

Python Practice Question Bank

50 Curated Coding Questions — Easy, Medium & Hard, with Full Solutions and Edge-Case Analysis

What's inside

- **50 questions total:** 17 Easy, 17 Medium, 16 Hard
- Every question includes a working, runnable Python solution with example calls
- Every solution is followed by **6 key points** covering time/space complexity, edge cases, common pitfalls, and alternative approaches
- Topics span strings, lists, dictionaries, recursion, dynamic programming, graphs, trees, heaps, and design patterns

Table of Contents

Easy (17 questions)

1. Reverse a String
2. Palindrome Check
3. Find Maximum in a List
4. Count Vowels
5. FizzBuzz
6. Prime Number Check
7. Sum of Digits
8. Factorial (Iterative)
9. Filter Even Numbers
10. Remove Duplicates (Order Preserved)
11. Swap Two Variables
12. Second Largest Number
13. Character Frequency Counter
14. Uppercase a List of Strings
15. Anagram Check
16. Sum a List Manually
17. Common Elements Between Two Lists

Medium (18 questions)

18. Fibonacci Sequence (Efficient)
19. Binary Search
20. Merge Two Sorted Lists
21. Find the Missing Number
22. Flatten a Nested List
23. Group Anagrams
24. Longest Substring Without Repeating Characters
25. Two Sum
26. Matrix Transpose
27. Balanced Parentheses
28. All Permutations of a String
29. Stack Implementation
30. Word Frequency Counter
31. Rotate Array
32. Intersection and Union of Two Lists
33. LRU-Style Ordered Cache
34. Chunk a List
35. Sort a Dictionary by Value

Hard (15 questions)

36. Detect Cycle in a Linked List
37. Merge K Sorted Lists
38. Longest Common Subsequence
39. Trie (Prefix Tree)
40. Dijkstra's Shortest Path

- 41. Quick Sort
- 42. Merge Overlapping Intervals
- 43. Kth Largest Element
- 44. Serialize and Deserialize a Binary Tree
- 45. Topological Sort
- 46. LRU Cache with Doubly Linked List ($O(1)$ from scratch)
- 47. Word Ladder (Shortest Transformation, BFS)
- 48. N-Queens Problem
- 49. Coin Change (Minimum Coins)
- 50. TTL Memoization Decorator

Easy Questions

Fundamentals: strings, lists, loops, and basic algorithms. Good for warm-ups and screening interviews.

Q1. Reverse a String

EASY

Problem: Write a function that reverses a given string.

```
def reverse_string(s):
    if not isinstance(s, str):
        raise TypeError("Input must be a string")
    return s[::-1]

print(reverse_string("hello")) # "olleh"
print(reverse_string(""))      # ""
print(reverse_string("a"))     # "a"
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(n)$, space: $O(n)$ for the new reversed string.
2. Edge case: empty string returns empty string, not an error.
3. Edge case: single character string returns itself unchanged.
4. Edge case: strings with unicode/emoji reverse correctly since slicing works on code points.
5. Pitfall: using a for-loop with string concatenation ($s = s + \text{char}$) is $O(n^2)$; slicing or list+join is preferred.
6. Alternative: `"".join(reversed(s))` or a two-pointer swap on a list(s) for an in-place style solution.

Q2. Palindrome Check

EASY

Problem: Check whether a given string is a palindrome, ignoring case and non-alphanumeric characters.

```
def is_palindrome(s):
    cleaned = [c.lower() for c in s if c.isalnum()]
    return cleaned == cleaned[::-1]

print(is_palindrome("A man, a plan, a canal: Panama")) # True
print(is_palindrome(""))                               # True
print(is_palindrome("ab"))                             # False
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(n)$ to clean + $O(n)$ to compare, overall $O(n)$.
2. Edge case: empty string is trivially a palindrome (returns True).
3. Edge case: strings with only punctuation/spaces reduce to an empty list -> True.
4. Edge case: case sensitivity handled via `.lower()`; 'Racecar' still passes.
5. Pitfall: forgetting to strip non-alphanumeric characters (spaces, commas) breaks real sentences.
6. Alternative: two-pointer approach scanning from both ends is $O(1)$ extra space (excluding cleaning).

Q3. Find Maximum in a List

EASY

Problem: Find the maximum value in a list of numbers without using the built-in `max()`.

```
def find_max(nums):
    if not nums:
        raise ValueError("List is empty")
    result = nums[0]
    for n in nums[1:]:
        if n > result:
            result = n
    return result

print(find_max([3, 7, 2, 9, 4])) # 9
print(find_max([-5, -1, -9])) # -1
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(n)$, single pass, space $O(1)$.
2. Edge case: empty list raises a clear `ValueError` instead of crashing obscurely.
3. Edge case: list with all negative numbers still returns the correct (least negative) max.
4. Edge case: list with a single element returns that element.
5. Pitfall: initializing `result = 0` instead of `nums[0]` fails for all-negative lists.
6. Alternative: `functools.reduce(lambda a, b: a if a > b else b, nums)` achieves the same in one line.

Q4. Count Vowels

EASY

Problem: Count the number of vowels (a, e, i, o, u) in a string, case-insensitive.

```
def count_vowels(s):
    vowels = set("aeiouAEIOU")
    return sum(1 for ch in s if ch in vowels)

print(count_vowels("Hello World")) # 3
print(count_vowels("")) # 0
print(count_vowels("XYZ")) # 0
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(n)$, space $O(1)$ (vowel set is constant size).
2. Edge case: empty string returns 0, not an error.
3. Edge case: strings with no vowels (e.g. 'xyz') return 0.
4. Edge case: mixed-case input is handled by including both cases in the vowel set.
5. Pitfall: using `.lower()` inside the loop repeatedly is fine performance-wise, but forgetting case handling entirely undercounts.
6. Alternative: use `re.findall('[aeiouAEIOU]', s)` and take `len()` of the result.

Q5. FizzBuzz

EASY

Problem: Print numbers 1 to n; multiples of 3 print 'Fizz', multiples of 5 print 'Buzz', multiples of both print 'FizzBuzz'.

```
def fizzbuzz(n):
    result = []
    for i in range(1, n + 1):
        if i % 15 == 0:
            result.append("FizzBuzz")
        elif i % 3 == 0:
            result.append("Fizz")
        elif i % 5 == 0:
            result.append("Buzz")
        else:
            result.append(str(i))
    return result

print(fizzbuzz(15))
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(n)$, space $O(n)$ for the result list.
2. Edge case: $n = 0$ or negative returns an empty list (range handles it gracefully).
3. Edge case: $n = 15$ correctly prints 'FizzBuzz' exactly once, not 'Fizz' then 'Buzz'.
4. Pitfall: checking %3 and %5 before %15 causes multiples of 15 to only ever print 'Fizz'.
5. Pitfall: forgetting `str(i)` for the else branch causes a mixed-type list (ints and strings).
6. Alternative: build the string incrementally: `out = ('Fizz' if i%3==0 else '') + ('Buzz' if i%5==0 else '') or str(i)`.

Q6. Prime Number Check

EASY

Problem: Determine whether a given integer is prime.

```
def is_prime(n):
    if n < 2:
        return False
    if n in (2, 3):
        return True
    if n % 2 == 0:
        return False
    for i in range(3, int(n ** 0.5) + 1, 2):
        if n % i == 0:
            return False
    return True

print(is_prime(17)) # True
print(is_prime(1)) # False
print(is_prime(2)) # True
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(\sqrt{n})$ thanks to checking divisors only up to the square root.
2. Edge case: $n < 2$ (0, 1, negative numbers) is correctly not prime.
3. Edge case: $n = 2$ is the only even prime and is handled explicitly.
4. Edge case: large n still runs fast because of the sqrt bound instead of checking all divisors.
5. Pitfall: looping `range(2, n)` instead of `range(2, sqrt(n)+1)` is correct but wastefully slow for large n .
6. Alternative: use the Sieve of Eratosthenes if checking primality for many numbers up to a limit at once.

Q7. Sum of Digits

EASY

Problem: Compute the sum of digits of a non-negative integer.

```
def sum_of_digits(n):
    if n < 0:
        raise ValueError("n must be non-negative")
    total = 0
    while n > 0:
        total += n % 10
        n //= 10
    return total

print(sum_of_digits(12345)) # 15
print(sum_of_digits(0))    # 0
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(\log_{10} n)$ since the loop runs once per digit, space $O(1)$.
2. Edge case: $n = 0$ returns 0 (the while loop simply never executes).
3. Edge case: negative input raises a `ValueError` instead of looping forever (since n never becomes 0).
4. Edge case: single-digit numbers return themselves.
5. Pitfall: using $n / 10$ instead of $n // 10$ in Python 3 produces a float and an infinite loop.
6. Alternative: `sum(int(d) for d in str(n))` is more concise but converts to a string, using extra memory.

Q8. Factorial (Iterative)

EASY

Problem: Compute $n!$ iteratively for a non-negative integer n .

```
def factorial(n):
    if n < 0:
        raise ValueError("Factorial undefined for negative numbers")
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

print(factorial(5)) # 120
print(factorial(0)) # 1
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(n)$, space $O(1)$ for the iterative version.
2. Edge case: $n = 0$ correctly returns 1 (empty product convention).
3. Edge case: $n = 1$ returns 1.
4. Edge case: negative input raises `ValueError` instead of returning a nonsensical value.
5. Pitfall: a naive recursive version without memoization risks `RecursionError` for large n (Python's default recursion limit ~ 1000).
6. Alternative: `math.factorial(n)` is the built-in, C-optimized equivalent for production code.

Q9. Filter Even Numbers

EASY

Problem: Return only the even numbers from a list, preserving order.

```
def filter_evens(nums):
    return [n for n in nums if n % 2 == 0]

print(filter_evens([1, 2, 3, 4, 5, 6])) # [2, 4, 6]
print(filter_evens([]))                 # []
print(filter_evens([-4, -3, -2]))       # [-4, -2]
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(n)$, space $O(k)$ where k is the number of even elements.
2. Edge case: empty list returns an empty list.
3. Edge case: negative even numbers are handled correctly since Python's `%` works with sign of divisor.
4. Edge case: list with no even numbers returns an empty list, not `None`.
5. Pitfall: using `n % 2 == 1` to test 'not odd' fails for negative odd numbers in some languages (not Python, but worth noting).
6. Alternative: `filter(lambda n: n % 2 == 0, nums)` works too but a list comprehension is more Pythonic/readable.

Q10. Remove Duplicates (Order Preserved)

EASY

Problem: Remove duplicate elements from a list while preserving the original order.

```
def remove_duplicates(items):
    seen = set()
    result = []
    for item in items:
        if item not in seen:
            seen.add(item)
            result.append(item)
    return result

print(remove_duplicates([1, 2, 2, 3, 1, 4])) # [1, 2, 3, 4]
print(remove_duplicates([]))                 # []
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(n)$ average, using a set for $O(1)$ average membership checks.
2. Edge case: empty list returns an empty list.
3. Edge case: list with all identical elements returns a single-element list.
4. Edge case: unhashable items (like lists) would raise a `TypeError` -- must be hashable for the 'seen' set.
5. Pitfall: using `dict.fromkeys(items)` is a cleaner one-liner (Python 3.7+ preserves insertion order) but changes behavior slightly for unhashable-adjacent cases.
6. Alternative: `list(dict.fromkeys(items))` achieves the same result more concisely.

Q11. Swap Two Variables

EASY

Problem: Swap the values of two variables without using a temporary variable.

```
def swap(a, b):  
    a, b = b, a  
    return a, b  
  
print(swap(1, 2))      # (2, 1)  
print(swap("x", "y"))  # ('y', 'x')
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(1)$, space $O(1)$ -- tuple packing/unpacking is a single bytecode-level operation.
2. Edge case: works for any object type (numbers, strings, lists) since Python swaps references, not values.
3. Edge case: swapping a variable with itself (`swap(a, a)`) is a no-op and safe.
4. Pitfall: in languages without tuple unpacking, forgetting a temp variable causes one value to be overwritten before use.
5. Pitfall: this returns a new tuple; if swapping in-place list elements, use `lst[i], lst[j] = lst[j], lst[i]`.
6. Alternative: arithmetic swap (`a = a+b; b = a-b; a = a-b`) works only for numbers and risks overflow -- avoid it.

Q12. Second Largest Number

EASY

Problem: Find the second largest distinct number in a list.

```
def second_largest(nums):  
    unique = list(set(nums))  
    if len(unique) < 2:  
        raise ValueError("Need at least 2 distinct numbers")  
    unique.sort(reverse=True)  
    return unique[1]  
  
print(second_largest([4, 1, 7, 7, 3])) # 4  
print(second_largest([5, 5, 5]))      # raises ValueError
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(n \log n)$ due to sorting the unique values, space $O(n)$.
2. Edge case: a list with all identical values has no second distinct value -- raises `ValueError`.
3. Edge case: duplicates of the max are ignored since we operate on a set first (e.g. `[7,7,4]` -> second largest is 4).
4. Edge case: a list of fewer than 2 elements also raises `ValueError`.
5. Pitfall: sorting the raw list (with duplicates) and taking index 1 gives the wrong answer when the max repeats.
6. Alternative: a single $O(n)$ pass tracking first and second max avoids the $O(n \log n)$ sort for large inputs.

Q13. Character Frequency Counter

EASY

Problem: Count how many times each character appears in a string.

```
from collections import Counter

def char_frequency(s):
    return dict(Counter(s))

print(char_frequency("banana"))
# {'b': 1, 'a': 3, 'n': 2}
print(char_frequency(""))
# {}
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(n)$, space $O(k)$ where k is the number of distinct characters.
2. Edge case: empty string returns an empty dict.
3. Edge case: whitespace and punctuation are counted as characters too unless explicitly filtered.
4. Edge case: case sensitivity means 'A' and 'a' are counted separately by default.
5. Pitfall: manually incrementing counts with `dict.get(ch, 0) + 1` works but Counter is more efficient and idiomatic.
6. Alternative: Counter also supports `.most_common(k)` to directly get the top-k frequent characters.

Q14. Uppercase a List of Strings

EASY

Problem: Convert every string in a list to uppercase.

```
def uppercase_all(strings):
    return [s.upper() for s in strings]

print(uppercase_all(["hello", "World", ""]))
# ['HELLO', 'WORLD', '']
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(\text{total characters across all strings})$, space $O(\text{same})$.
2. Edge case: empty strings in the list remain empty strings after uppercasing.
3. Edge case: an empty input list returns an empty list.
4. Edge case: strings already fully uppercase, or containing numbers/symbols, are unaffected/unchanged appropriately.
5. Pitfall: calling `.upper()` on a non-string element (e.g. an int in the list) raises `AttributeError` -- validate types if input is untrusted.
6. Alternative: `map(str.upper, strings)` is equally valid; wrap in `list()` to materialize it.

Q15. Anagram Check

EASY

Problem: Determine if two strings are anagrams of each other.

```
def is_anagram(s1, s2):
    s1 = s1.replace(" ", "").lower()
    s2 = s2.replace(" ", "").lower()
    return sorted(s1) == sorted(s2)

print(is_anagram("listen", "silent")) # True
print(is_anagram("Dormitory", "Dirty Room")) # True
print(is_anagram("abc", "abcd")) # False
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(n \log n)$ due to sorting both strings; space $O(n)$.
2. Edge case: strings of different lengths (after removing spaces) can never be anagrams; `sorted()` comparison handles this naturally.
3. Edge case: case-insensitivity handled via `.lower()`.
4. Edge case: two empty strings are trivially anagrams of each other (True).
5. Pitfall: forgetting to strip spaces breaks multi-word phrase comparisons like 'Dormitory' vs 'Dirty Room'.
6. Alternative: `Counter(s1) == Counter(s2)` achieves $O(n)$ instead of $O(n \log n)$ since it avoids sorting.

Q16. Sum a List Manually

EASY

Problem: Compute the sum of a list of numbers without using the built-in `sum()`.

```
def manual_sum(nums):
    total = 0
    for n in nums:
        total += n
    return total

print(manual_sum([1, 2, 3, 4])) # 10
print(manual_sum([])) # 0
print(manual_sum([-1, 1])) # 0
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(n)$, space $O(1)$.
2. Edge case: empty list correctly returns 0 (the identity for addition).
3. Edge case: list of floats sums correctly, though floating point rounding may apply.
4. Edge case: mixed negative and positive numbers cancel out correctly.
5. Pitfall: initializing `total` as an `int 0` when summing a list of `Decimal` or complex numbers can raise a `TypeError` -- consider `total = nums[0]*0` pattern for generality.
6. Alternative: `functools.reduce(operator.add, nums, 0)` is a functional-style equivalent.

Q17. Common Elements Between Two Lists

EASY

Problem: Find the common elements between two lists (no duplicates in output).

```
def common_elements(a, b):  
    return list(set(a) & set(b))  
  
print(sorted(common_elements([1, 2, 3], [2, 3, 4]))) # [2, 3]  
print(common_elements([], [1, 2]))                 # []
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(n + m)$ using set intersection, space $O(n + m)$.
2. Edge case: one or both lists empty returns an empty list.
3. Edge case: lists with no overlap return an empty list.
4. Edge case: result order is not guaranteed since sets are unordered -- sort the output if order matters.
5. Pitfall: this approach silently removes duplicate common values; if counts matter (e.g. $[1,1,2]$ & $[1,2,2]$ should give two 1's), use Counter intersection instead.
6. Alternative: `(Counter(a) & Counter(b)).elements()` preserves multiplicities for multiset intersection.

Medium Questions

Data structures and common interview patterns: hashing, sliding window, sorting, stacks, and caching.

Q18. Fibonacci Sequence (Efficient)

MEDIUM

Problem: Compute the nth Fibonacci number efficiently for large n.

```
def fibonacci(n):
    if n < 0:
        raise ValueError("n must be non-negative")
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a

print(fibonacci(10)) # 55
print(fibonacci(0)) # 0
print(fibonacci(1)) # 1
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(n)$, space $O(1)$ -- avoids the exponential blow-up of naive recursion.
2. Edge case: $n = 0$ returns 0, $n = 1$ returns 1 (base cases handled by the loop naturally).
3. Edge case: negative n raises a `ValueError` rather than looping incorrectly.
4. Edge case: very large n (e.g. 10000) still works because Python integers have arbitrary precision.
5. Pitfall: naive recursive `fibonacci(n) = fibonacci(n-1) + fibonacci(n-2)` is $O(2^n)$ and becomes unusably slow past $n \sim 35$.
6. Alternative: `functools.lru_cache` on a recursive version gives $O(n)$ time with $O(n)$ recursion-stack space.

Q19. Binary Search

MEDIUM

Problem: Implement binary search on a sorted list, returning the index of the target or -1.

```
def binary_search(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1

print(binary_search([1, 3, 5, 7, 9], 7)) # 3
print(binary_search([1, 3, 5], 4))      # -1
print(binary_search([], 1))             # -1
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(\log n)$, space $O(1)$ (iterative version).
2. Edge case: empty array immediately returns -1 since the while loop never executes.
3. Edge case: target smaller than all elements or larger than all elements correctly returns -1.
4. Edge case: array with a single element works correctly for both match and no-match.
5. Pitfall: using `mid = (lo + hi) // 2` can overflow in fixed-width-integer languages (not an issue in Python, but worth knowing); prefer `lo + (hi - lo) // 2` as a language-agnostic habit.
6. Pitfall: the input MUST be sorted; running binary search on unsorted data silently gives wrong results without erroring.

Q20. Merge Two Sorted Lists

MEDIUM

Problem: Merge two sorted lists into a single sorted list.

```
def merge_sorted(a, b):
    i, j, merged = 0, 0, []
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            merged.append(a[i]); i += 1
        else:
            merged.append(b[j]); j += 1
    merged.extend(a[i:])
    merged.extend(b[j:])
    return merged

print(merge_sorted([1, 3, 5], [2, 4, 6])) # [1, 2, 3, 4, 5, 6]
print(merge_sorted([], [1, 2]))         # [1, 2]
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(n + m)$, space $O(n + m)$ for the merged output.
2. Edge case: one list empty simply returns the other list's contents in order.
3. Edge case: both lists empty returns an empty list.
4. Edge case: duplicate values across lists are preserved (stable merge, $\leq$ keeps 'a' first on ties).
5. Pitfall: forgetting the two `extend()` calls at the end drops the remaining tail of whichever list wasn't exhausted.
6. Alternative: `heapq.merge(a, b)` is a built-in, memory-efficient (iterator-based) way to merge sorted iterables.

Q21. Find the Missing Number

MEDIUM

Problem: Given a list containing n distinct numbers from 0 to n , find the missing number.

```
def missing_number(nums):
    n = len(nums)
    expected_sum = n * (n + 1) // 2
    return expected_sum - sum(nums)

print(missing_number([3, 0, 1])) # 2
print(missing_number([0, 1]))   # 2
print(missing_number([9,6,4,2,3,5,7,0,1])) # 8
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(n)$, space $O(1)$ using Gauss's summation formula.
2. Edge case: the missing number could be 0 or n itself (the boundary values) -- the formula handles both.
3. Edge case: a list of length 1, e.g. `[0]` or `[1]`, correctly identifies the missing value (1 or 0).
4. Edge case: assumes exactly one number is missing and all others are distinct and in range -- unvalidated input could give a wrong silent answer.
5. Pitfall: for very large n , `expected_sum` could overflow in other languages; Python's arbitrary-precision ints avoid this.
6. Alternative: XOR-based approach (XOR all indices $0..n$ with all values) avoids any risk of numeric overflow entirely.

Q22. Flatten a Nested List

MEDIUM

Problem: Flatten an arbitrarily nested list into a single flat list.

```
def flatten(lst):
    result = []
    for item in lst:
        if isinstance(item, list):
            result.extend(flatten(item))
        else:
            result.append(item)
    return result

print(flatten([1, [2, 3, [4, 5]], 6])) # [1, 2, 3, 4, 5, 6]
print(flatten([])) # []
print(flatten([1, [], [2, [3]]])) # [1, 2, 3]
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(n)$ where n is the total number of elements across all nesting levels.
2. Edge case: empty list, or lists containing empty sublists, are handled without error.
3. Edge case: deeply nested lists work via recursion, though extremely deep nesting risks Python's recursion limit.
4. Edge case: non-list iterables like tuples or strings are treated as scalar items, not flattened -- clarify requirements if tuples should also flatten.
5. Pitfall: using `isinstance(item, (list, tuple))` instead of just list changes behavior -- be explicit about which types count as 'nested'.
6. Alternative: an iterative version using an explicit stack avoids recursion-depth issues for very deep structures.

Q23. Group Anagrams

MEDIUM

Problem: Group a list of strings into sets of anagrams.

```
from collections import defaultdict

def group_anagrams(words):
    groups = defaultdict(list)
    for w in words:
        key = "".join(sorted(w))
        groups[key].append(w)
    return list(groups.values())

print(group_anagrams(["eat", "tea", "tan", "ate", "nat", "bat"]))
# [['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(n * k \log k)$ where n = number of words, k = max word length (sorting each word).
2. Edge case: empty input list returns an empty list of groups.
3. Edge case: an empty string "" is its own anagram group (`sorted("") == ""`).
4. Edge case: words with different cases ('Eat' vs 'eat') are NOT grouped together unless normalized -- clarify case sensitivity requirements.
5. Pitfall: using a plain dict instead of defaultdict requires manual 'if key not in groups' checks -- easy to forget.
6. Alternative: using a character-count tuple as the key (26-length tuple for lowercase letters) achieves $O(n * k)$ instead of $O(n * k \log k)$.

Q24. Longest Substring Without Repeating Characters

MEDIUM

Problem: Find the length of the longest substring without repeating characters.

```
def longest_unique_substring(s):
    seen = {}
    start = max_len = 0
    for i, ch in enumerate(s):
        if ch in seen and seen[ch] >= start:
            start = seen[ch] + 1
        seen[ch] = i
        max_len = max(max_len, i - start + 1)
    return max_len

print(longest_unique_substring("abcabcbb")) # 3
print(longest_unique_substring("")) # 0
print(longest_unique_substring("bbbb")) # 1
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(n)$ using a sliding window with a hash map, space $O(\min(n, \text{alphabet size}))$.
2. Edge case: empty string returns 0.
3. Edge case: string with all identical characters (e.g. 'bbbb') returns 1.
4. Edge case: string where all characters are unique returns $\text{len}(s)$.
5. Pitfall: forgetting the ' $\text{seen}[ch] \geq \text{start}$ ' check causes the window to incorrectly shrink based on stale (out-of-window) character positions.
6. Alternative: a brute-force check of every substring is $O(n^3)$ and only suitable for very small inputs or teaching purposes.

Q25. Two Sum

MEDIUM

Problem: Given a list of numbers and a target, return the indices of the two numbers that add up to the target.

```
def two_sum(nums, target):
    seen = {}
    for i, n in enumerate(nums):
        complement = target - n
        if complement in seen:
            return [seen[complement], i]
        seen[n] = i
    return []

print(two_sum([2, 7, 11, 15], 9)) # [0, 1]
print(two_sum([3, 3], 6)) # [0, 1]
print(two_sum([1, 2], 10)) # []
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(n)$, space $O(n)$ using a single-pass hash map.
2. Edge case: no valid pair exists -- returns an empty list rather than raising an error (define this behavior clearly upfront).
3. Edge case: duplicate values that sum to the target (e.g. [3,3], target=6) work correctly because we check 'seen' before inserting the current number.
4. Edge case: negative numbers and zero are handled correctly since dict lookups don't care about sign.
5. Pitfall: inserting into 'seen' BEFORE checking the complement could incorrectly match a number with itself for a single occurrence.
6. Alternative: brute-force nested loop is $O(n^2)$ but avoids extra space and is fine for very small inputs.

Q26. Matrix Transpose

MEDIUM

Problem: Transpose a 2D matrix (rows become columns).

```
def transpose(matrix):
    if not matrix:
        return []
    return [list(row) for row in zip(*matrix)]

print(transpose([[1, 2, 3], [4, 5, 6]]))
# [[1, 4], [2, 5], [3, 6]]
print(transpose([]))
# []
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(\text{rows} * \text{cols})$, space $O(\text{rows} * \text{cols})$ for the new matrix.
2. Edge case: empty matrix ([]) returns an empty list.
3. Edge case: a matrix with a single row becomes a matrix with a single column, and vice versa.
4. Edge case: non-rectangular (ragged) input rows cause `zip()` to silently truncate to the shortest row -- validate row lengths if strict rectangularity is required.
5. Pitfall: using nested loops with manual index math (`result[j][i] = matrix[i][j]`) requires pre-allocating the result matrix correctly sized, which is error-prone.
6. Alternative: with NumPy, `matrix.T` or `np.transpose(matrix)` is the vectorized, high-performance equivalent for large matrices.

Q27. Balanced Parentheses

MEDIUM

Problem: Check whether a string of brackets () [] { } is balanced.

```
def is_balanced(s):
    pairs = {'(': ')', '[': ']', '{': '}'
    stack = []
    for ch in s:
        if ch in "([{":
            stack.append(ch)
        elif ch in ")]}":
            if not stack or stack.pop() != pairs[ch]:
                return False
    return not stack

print(is_balanced("({[]})")) # True
print(is_balanced("]"))      # False
print(is_balanced(""))       # True
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(n)$, space $O(n)$ worst case (all opening brackets).
2. Edge case: empty string is considered balanced (True) by convention.
3. Edge case: a string with only closing brackets (e.g. `)]}]`) correctly returns False since the stack is empty when popping.
4. Edge case: mismatched types in correct-looking nesting (e.g. `{(})`) is correctly caught by the pairs dictionary check.
5. Pitfall: forgetting the final `'return not stack'` check misses unclosed opening brackets left on the stack (e.g. `{{{()`).
6. Pitfall: non-bracket characters in the string are silently ignored here -- decide if the function should validate only brackets or reject other characters too.

Q28. All Permutations of a String

MEDIUM

Problem: Generate all permutations of a given string.

```
from itertools import permutations

def all_permutations(s):
    return ["".join(p) for p in permutations(s)]

print(all_permutations("abc"))
# ['abc', 'acb', 'bac', 'bca', 'cab', 'cba']
print(all_permutations(""))
# ['']
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(n! * n)$ -- there are $n!$ permutations, each of length n to join.
2. Edge case: empty string returns a list containing one empty string [""].
3. Edge case: single character string returns a single-element list, e.g. ['a'].
4. Edge case: repeated characters in the string (e.g. 'aab') produce duplicate permutations -- dedupe with `set(...)` if uniqueness is required.
5. Pitfall: for longer strings ($n > \sim 10$), $n!$ grows explosively ($10! = 3,628,800$) -- this approach doesn't scale to large n .
6. Alternative: write a manual recursive/backtracking permutation generator to avoid the `itertools` dependency or to prune duplicates early for repeated characters.

Q29. Stack Implementation

MEDIUM

Problem: Implement a Stack class with push, pop, peek, and is_empty operations.

```
class Stack:
    def __init__(self):
        self._items = []

    def push(self, item):
        self._items.append(item)

    def pop(self):
        if self.is_empty():
            raise IndexError("pop from empty stack")
        return self._items.pop()

    def peek(self):
        if self.is_empty():
            raise IndexError("peek from empty stack")
        return self._items[-1]

    def is_empty(self):
        return len(self._items) == 0

s = Stack()
s.push(1); s.push(2)
print(s.pop())    # 2
print(s.is_empty()) # False
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(1)$ amortized for push/pop/peek using a Python list as the backing store.
2. Edge case: popping or peeking an empty stack raises a clear `IndexError` instead of returning `None` silently.
3. Edge case: pushing many items works fine since Python lists dynamically resize (amortized $O(1)$ append).
4. Edge case: `is_empty()` correctly returns `True` right after initialization and after popping the last item.
5. Pitfall: using `list.pop(0)` or `insert(0, ...)` instead of `append/pop(-1)` turns operations into $O(n)$ -- always operate on the end of the list.
6. Alternative: `collections.deque` is preferred over a list when you need a stack AND queue-like operations from both ends efficiently.

Q30. Word Frequency Counter

MEDIUM

Problem: Count the frequency of each word in a block of text, case-insensitive, ignoring punctuation.

```
import re
from collections import Counter

def word_frequency(text):
    words = re.findall(r'[a-z0-9]+', text.lower())
    return dict(Counter(words))

print(word_frequency("The cat sat. The CAT ran!"))
# {'the': 2, 'cat': 2, 'sat': 1, 'ran': 1}
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(n)$ where n is the length of the text, using regex tokenization + Counter.
2. Edge case: empty text returns an empty dict.
3. Edge case: punctuation attached to words (e.g. 'ran!', 'sat.') is stripped correctly by the regex.
4. Edge case: case differences ('CAT' vs 'cat') are unified via `.lower()` before counting.
5. Pitfall: using `text.split()` alone (no regex) leaves punctuation stuck to words, so 'cat' and 'cat!' would be counted as different words.
6. Alternative: for advanced tokenization (contractions, hyphenated words, unicode text), use a proper NLP library like `nltk` or `spaCy`.

Q31. Rotate Array

MEDIUM

Problem: Rotate a list to the right by k positions.

```
def rotate(nums, k):
    if not nums:
        return nums
    n = len(nums)
    k = k % n
    return nums[-k:] + nums[:-k] if k else nums[:]

print(rotate([1, 2, 3, 4, 5], 2)) # [4, 5, 1, 2, 3]
print(rotate([1, 2, 3], 0))      # [1, 2, 3]
print(rotate([1, 2, 3], 5))      # [2, 3, 1] (5 % 3 = 2)
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(n)$, space $O(n)$ for the new rotated list ($O(1)$ extra if rotating in place with reversal).
2. Edge case: empty list returns itself unchanged.
3. Edge case: $k = 0$ returns a copy of the original order.
4. Edge case: k larger than `len(nums)` is handled correctly via the modulo operation ($k \% n$).
5. Pitfall: forgetting ' $k \% n$ ' causes slicing errors or incorrect rotation amounts when $k > n$.
6. Alternative: an in-place $O(1)$ -space solution uses the 'reverse three times' trick: reverse the whole array, then reverse each of the two segments.

Q32. Intersection and Union of Two Lists

MEDIUM

Problem: Compute both the intersection and union of two lists, without duplicates.

```
def intersection_union(a, b):
    set_a, set_b = set(a), set(b)
    return sorted(set_a & set_b), sorted(set_a | set_b)

inter, uni = intersection_union([1, 2, 2, 3], [2, 3, 4])
print(inter) # [2, 3]
print(uni)   # [1, 2, 3, 4]
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(n + m)$, space $O(n + m)$ for the resulting sets.
2. Edge case: one list empty makes intersection empty and union equal to the other list's unique elements.
3. Edge case: identical lists give intersection == union == the list's unique elements.
4. Edge case: unsortable/mixed types (e.g. strings and ints together) would raise a `TypeError` on `sorted()` -- ensure consistent types.
5. Pitfall: converting to a set silently drops duplicates, which is desired here but must be an explicit requirement, not an accident.
6. Alternative: if multiplicities matter, use `collections.Counter` with `&` and `|` operators for proper multiset intersection/union.

Q33. LRU-Style Ordered Cache

MEDIUM

Problem: Implement a fixed-size cache that evicts the least-recently-used item when full.

```
from collections import OrderedDict

class LRUCache:
    def __init__(self, capacity):
        if capacity <= 0:
            raise ValueError("capacity must be positive")
        self.capacity = capacity
        self.cache = OrderedDict()

    def get(self, key):
        if key not in self.cache:
            return -1
        self.cache.move_to_end(key)
        return self.cache[key]

    def put(self, key, value):
        if key in self.cache:
            self.cache.move_to_end(key)
        self.cache[key] = value
        if len(self.cache) > self.capacity:
            self.cache.popitem(last=False)

c = LRUCache(2)
c.put(1, 'a'); c.put(2, 'b')
c.get(1)       # 'a', marks 1 as recently used
c.put(3, 'c')   # evicts key 2 (least recently used)
print(c.get(2)) # -1
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(1)$ for get/put using `OrderedDict`'s `move_to_end` and `popitem`.
2. Edge case: `capacity <= 0` raises `ValueError` at construction time.
3. Edge case: getting a non-existent key returns `-1` (sentinel) instead of raising `KeyError`.
4. Edge case: updating an existing key's value refreshes its recency without growing the cache size.
5. Pitfall: forgetting `move_to_end()` on `get()` means 'recently used' tracking silently breaks, causing wrong eviction order.
6. Alternative: without `OrderedDict`, a dict (Python 3.7+ preserves insertion order) plus manual `pop`+`reinsert` achieves the same effect.

Q34. Chunk a List

MEDIUM

Problem: Split a list into sublists (chunks) of a given size.

```
def chunk_list(lst, size):
    if size <= 0:
        raise ValueError("size must be positive")
    return [lst[i:i + size] for i in range(0, len(lst), size)]

print(chunk_list([1,2,3,4,5,6,7], 3))
# [[1, 2, 3], [4, 5, 6], [7]]
print(chunk_list([], 3))
# []
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(n)$, space $O(n)$ for the resulting chunks.
2. Edge case: empty list returns an empty list of chunks.
3. Edge case: size larger than $\text{len}(\text{lst})$ returns a single chunk containing the whole list.
4. Edge case: the last chunk may be smaller than 'size' if $\text{len}(\text{lst})$ isn't evenly divisible.
5. Pitfall: $\text{size} \leq 0$ would cause $\text{range}(0, \text{len}(\text{lst}), \text{size})$ to raise a `ValueError` or loop forever -- explicitly validated here.
6. Alternative: `itertools`-based approaches (e.g. `more_itertools.chunked`) exist but require a third-party dependency; this pure-slicing approach needs none.

Q35. Sort a Dictionary by Value

MEDIUM

Problem: Sort a dictionary's items by value, descending, using a custom key function.

```
def sort_by_value(d, descending=True):
    return sorted(d.items(), key=lambda item: item[1], reverse=descending)

scores = {"alice": 88, "bob": 95, "carol": 72}
print(sort_by_value(scores))
# [('bob', 95), ('alice', 88), ('carol', 72)]
print(sort_by_value({}))
# []
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(n \log n)$ due to sorting, space $O(n)$ for the list of items.
2. Edge case: empty dictionary returns an empty list.
3. Edge case: tied values retain their relative original order since Python's sort is stable.
4. Edge case: `descending=False` correctly sorts ascending instead.
5. Pitfall: sorting `d.keys()` by value directly isn't possible without a key function referencing `d[k]` -- `item[1]` on `.items()` is cleaner and avoids repeated dict lookups.
6. Alternative: for only the top-k entries, `heapq.nlargest(k, d.items(), key=lambda x: x[1])` is more efficient than a full sort.

Hard Questions

Advanced algorithms: graphs, dynamic programming, trees, heaps, and backtracking. Suited for senior-level interviews.

Q36. Detect Cycle in a Linked List

HARD

Problem: Given the head of a singly linked list, determine if it contains a cycle.

```
class Node:
    def __init__(self, val):
        self.val = val
        self.next = None

def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow is fast:
            return True
    return False

# Example: build a -> b -> c -> a (cycle)
a, b, c = Node(1), Node(2), Node(3)
a.next, b.next, c.next = b, c, a
print(has_cycle(a))    # True
print(has_cycle(None)) # False
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(n)$, space $O(1)$ using Floyd's Tortoise and Hare algorithm.
2. Edge case: an empty list ($\text{head} = \text{None}$) returns False immediately.
3. Edge case: a single node with no self-loop returns False; a single node pointing to itself returns True.
4. Edge case: a long list with no cycle terminates correctly when fast or fast.next becomes None.
5. Pitfall: using a set to track visited nodes also works ($O(n)$ space) but Floyd's algorithm is preferred for $O(1)$ space.
6. Alternative: to also find the cycle's START node, after detecting a cycle reset one pointer to head and advance both one step at a time until they meet again.

Q37. Merge K Sorted Lists

HARD

Problem: Merge k sorted lists (as Python lists) into one sorted list.

```
import heapq

def merge_k_sorted(lists):
    heap = []
    for i, lst in enumerate(lists):
        if lst:
            heapq.heappush(heap, (lst[0], i, 0))
    result = []
    while heap:
        val, list_idx, elem_idx = heapq.heappop(heap)
        result.append(val)
        if elem_idx + 1 < len(lists[list_idx]):
            nxt = lists[list_idx][elem_idx + 1]
            heapq.heappush(heap, (nxt, list_idx, elem_idx + 1))
    return result

print(merge_k_sorted([[1,4,5],[1,3,4],[2,6]]))
# [1, 1, 2, 3, 4, 4, 5, 6]
print(merge_k_sorted([]))
# []
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(N \log k)$ where N is total elements and k is the number of lists, using a min-heap.
2. Edge case: empty outer list (no lists at all) returns an empty result.
3. Edge case: some individual lists being empty are skipped safely when building the initial heap.
4. Edge case: all lists containing identical/overlapping values are merged correctly and stably (tie-broken by list index).
5. Pitfall: pushing raw values without a tiebreaker (like `list_idx`) into the heap can raise a `TypeError` if values are unorderable/equal and Python tries to compare the next tuple element (lists themselves).
6. Alternative: repeatedly using `heapq.merge(*lists)` is a simpler built-in one-liner for already-sorted iterables, achieving the same complexity.

Q38. Longest Common Subsequence

HARD

Problem: Find the length of the longest common subsequence (LCS) between two strings.

```
def lcs_length(a, b):
    m, n = len(a), len(b)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if a[i-1] == b[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    return dp[m][n]

print(lcs_length("abcde", "ace")) # 3
print(lcs_length("", "abc"))      # 0
print(lcs_length("abc", "abc"))   # 3
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(m*n)$, space $O(m*n)$ for the DP table.
2. Edge case: either string empty gives LCS length 0.
3. Edge case: identical strings give LCS length equal to the full string length.
4. Edge case: strings with no characters in common at all give LCS length 0.
5. Pitfall: off-by-one errors in the DP table indices (using `a[i]` instead of `a[i-1]`) are extremely common -- the table is 1-indexed while the strings are 0-indexed.
6. Alternative: space can be optimized to $O(\min(m,n))$ by keeping only the previous DP row instead of the full 2D table, since each row only depends on the row above it.

Q39. Trie (Prefix Tree)

HARD

Problem: Implement a Trie supporting insert, search (exact word), and starts_with (prefix) operations.

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        node = self.root
        for ch in word:
            node = node.children.setdefault(ch, TrieNode())
        node.is_end = True

    def search(self, word):
        node = self._find(word)
        return node is not None and node.is_end

    def starts_with(self, prefix):
        return self._find(prefix) is not None

    def _find(self, s):
        node = self.root
        for ch in s:
            if ch not in node.children:
                return None
            node = node.children[ch]
        return node

t = Trie()
t.insert("apple")
print(t.search("apple"))      # True
print(t.search("app"))       # False
print(t.starts_with("app"))   # True
print(t.search(""))          # False
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(L)$ for insert/search/starts_with, where L is the length of the word/prefix.
2. Edge case: searching for an empty string returns False (unless "" was explicitly inserted).
3. Edge case: searching for a prefix that was never inserted as a full word (e.g. 'app' when only 'apple' inserted) correctly returns False for search() but True for starts_with().
4. Edge case: inserting the same word twice is idempotent -- is_end simply gets set to True again with no error.
5. Pitfall: confusing search() (must be a complete word) with starts_with() (any prefix match) is the most common bug when using a Trie.
6. Alternative: for memory efficiency on large alphabets, use a fixed-size array of 26 children instead of a dict when characters are limited to lowercase a-z.

Q40. Dijkstra's Shortest Path

HARD

Problem: Find the shortest path distances from a source node in a weighted graph with non-negative edge weights.

```
import heapq

def dijkstra(graph, source):
    dist = {node: float('inf') for node in graph}
    dist[source] = 0
    pq = [(0, source)]
    visited = set()

    while pq:
        d, node = heapq.heappop(pq)
        if node in visited:
            continue
        visited.add(node)
        for neighbor, weight in graph[node]:
            if d + weight < dist[neighbor]:
                dist[neighbor] = d + weight
                heapq.heappush(pq, (dist[neighbor], neighbor))
    return dist

graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('C', 2), ('D', 5)],
    'C': [('D', 1)],
    'D': []
}
print(dijkstra(graph, 'A'))
# {'A': 0, 'B': 1, 'C': 3, 'D': 4}
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O((V + E) \log V)$ using a binary heap priority queue.
2. Edge case: unreachable nodes remain at `float('inf')` in the result, correctly signaling 'no path'.
3. Edge case: the source node itself has distance 0.
4. Edge case: a graph with no edges (only isolated nodes) leaves every non-source node at infinity.
5. Pitfall: Dijkstra's algorithm gives WRONG results with negative edge weights -- use Bellman-Ford instead if negative weights are possible.
6. Pitfall: forgetting the 'visited' check allows stale, already-finalized nodes to be reprocessed, hurting performance (though not correctness, since we check 'if $d + \text{weight} < \text{dist}[\text{neighbor}]$ ').

Q41. Quick Sort

HARD

Problem: Implement quicksort to sort a list of numbers in ascending order.

```
def quicksort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    mid = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quicksort(left) + mid + quicksort(right)

print(quicksort([5, 2, 9, 1, 5, 6])) # [1, 2, 5, 5, 6, 9]
print(quicksort([]))                # []
print(quicksort([1]))                # [1]
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(n \log n)$ average, $O(n^2)$ worst case (e.g. already-sorted input with a poor pivot choice); space $O(n)$ for this non-in-place version.
2. Edge case: empty list and single-element list are returned immediately as base cases.
3. Edge case: a list with many duplicate values is handled correctly and efficiently via the explicit 'mid' partition (avoids the classic Dutch-flag duplicate problem).
4. Edge case: an already-sorted or reverse-sorted list still works correctly, though picking the middle element as pivot helps avoid worst-case behavior compared to always picking the first/last element.
5. Pitfall: this implementation is NOT in-place (uses extra list comprehensions), so it's not memory-optimal for very large arrays.
6. Alternative: an in-place Lomuto or Hoare partition scheme sorts with $O(\log n)$ extra space (recursion stack) instead of $O(n)$.

Q42. Merge Overlapping Intervals

HARD

Problem: Given a list of intervals, merge all overlapping intervals.

```
def merge_intervals(intervals):
    if not intervals:
        return []
    intervals = sorted(intervals, key=lambda x: x[0])
    merged = [intervals[0]]
    for start, end in intervals[1:]:
        last_start, last_end = merged[-1]
        if start <= last_end:
            merged[-1] = (last_start, max(last_end, end))
        else:
            merged.append((start, end))
    return merged

print(merge_intervals([(1,3), (2,6), (8,10), (15,18)]))
# [(1, 6), (8, 10), (15, 18)]
print(merge_intervals([]))
# []
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(n \log n)$ due to sorting by start time; space $O(n)$ for the result.
2. Edge case: empty interval list returns an empty list.
3. Edge case: a single interval returns a list containing just that interval, unmodified.
4. Edge case: intervals that only 'touch' at endpoints (e.g. (1,3) and (3,5)) are merged since $\text{start} \leq \text{last_end}$ includes equality.
5. Pitfall: forgetting to sort by start time first causes completely incorrect merges since the algorithm assumes sequential, sorted processing.
6. Pitfall: using $\max(\text{last_end}, \text{end})$ instead of just 'end' is crucial -- a fully-nested interval (e.g. (1,10) then (2,3)) must not shrink the merged interval's end.

Q43. Kth Largest Element

HARD

Problem: Find the kth largest element in an unsorted list.

```
import heapq

def kth_largest(nums, k):
    if k < 1 or k > len(nums):
        raise ValueError("k out of range")
    return heapq.nlargest(k, nums)[-1]

print(kth_largest([3, 2, 1, 5, 6, 4], 2)) # 5
print(kth_largest([1], 1))               # 1
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(n \log k)$ using a heap of size k , much better than a full $O(n \log n)$ sort for small k .
2. Edge case: k out of valid range ($k < 1$ or $k > \text{len}(\text{nums})$) raises a clear `ValueError`.
3. Edge case: $k = 1$ returns the maximum element; $k = \text{len}(\text{nums})$ returns the minimum.
4. Edge case: duplicate values are counted by position, not uniqueness -- e.g. `kth_largest([1,1,2], 2)` is 1, not skipping the duplicate.
5. Pitfall: sorting the entire list (`sorted(nums)[-k]`) works but is $O(n \log n)$ -- wasteful when k is small relative to n .
6. Alternative: the Quickselect algorithm achieves average $O(n)$ time (worst-case $O(n^2)$) and is the standard interview-optimal solution.

Q44. Serialize and Deserialize a Binary Tree

HARD

Problem: Convert a binary tree to a string and back, preserving structure.

```
class TreeNode:
    def __init__(self, val, left=None, right=None):
        self.val, self.left, self.right = val, left, right

def serialize(root):
    if root is None:
        return "#"
    return f"{root.val},{serialize(root.left)},{serialize(root.right)}"

def deserialize(data):
    values = iter(data.split(","))
    def build():
        val = next(values)
        if val == "#":
            return None
        node = TreeNode(int(val))
        node.left = build()
        node.right = build()
        return node
    return build()

root = TreeNode(1, TreeNode(2), TreeNode(3, TreeNode(4)))
s = serialize(root)
print(s)  # "1,2,##,3,4,##,"
restored = deserialize(s)
print(restored.right.left.val)  # 4
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(n)$ for both serialize and deserialize, visiting each node once.
2. Edge case: an empty tree ($root = None$) serializes to just '#' and deserializes back to None.
3. Edge case: a tree with only a root node (no children) serializes correctly with two '#' markers for its null children.
4. Edge case: a skewed tree (every node has only a left or only a right child) still round-trips correctly since preorder + null markers fully capture structure.
5. Pitfall: using an inorder traversal alone for serialization is ambiguous (multiple trees can share the same inorder sequence) -- preorder with explicit null markers avoids this.
6. Alternative: level-order (BFS) serialization is more common in practice (e.g. LeetCode's format) and avoids deep recursion for very tall trees.

Q45. Topological Sort

HARD

Problem: Given a directed acyclic graph (DAG), return a valid topological ordering of its nodes.

```
from collections import deque, defaultdict

def topological_sort(num_nodes, edges):
    graph = defaultdict(list)
    indegree = [0] * num_nodes
    for u, v in edges:
        graph[u].append(v)
        indegree[v] += 1

    queue = deque([n for n in range(num_nodes) if indegree[n] == 0])
    order = []
    while queue:
        node = queue.popleft()
        order.append(node)
        for neighbor in graph[node]:
            indegree[neighbor] -= 1
            if indegree[neighbor] == 0:
                queue.append(neighbor)

    if len(order) != num_nodes:
        raise ValueError("Graph has a cycle -- no valid topological order")
    return order

print(topological_sort(4, [(0,1), (0,2), (1,3), (2,3)]))
# [0, 1, 2, 3]
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(V + E)$ using Kahn's algorithm (BFS with indegree tracking).
2. Edge case: a graph with no edges returns nodes in any order (all indegree 0 initially, order determined by queue insertion).
3. Edge case: disconnected components are all still included in the final order.
4. Edge case: a graph containing a cycle is detected because not all nodes get added to 'order' -- raises `ValueError` instead of returning a partial/wrong result silently.
5. Pitfall: forgetting to check `len(order) != num_nodes` at the end means cyclic graphs silently produce an incomplete, misleading ordering.
6. Alternative: a DFS-based approach (postorder + reverse) also produces a valid topological order and is often used when a full traversal is more natural than layer-by-layer processing.

Q46. LRU Cache with Doubly Linked List (O(1) from scratch)

HARD

Problem: Implement an LRU cache with true O(1) get/put using a hash map + doubly linked list (no OrderedDict).

```
class DNode:
    def __init__(self, key=None, val=None):
        self.key, self.val = key, val
        self.prev = self.next = None

class LRUCache:
    def __init__(self, capacity):
        self.capacity = capacity
        self.map = {}
        self.head = DNode() # dummy head (most recent)
        self.tail = DNode() # dummy tail (least recent)
        self.head.next, self.tail.prev = self.tail, self.head

    def _remove(self, node):
        node.prev.next, node.next.prev = node.next, node.prev

    def _add_front(self, node):
        node.next = self.head.next
        node.prev = self.head
        self.head.next.prev = node
        self.head.next = node

    def get(self, key):
        if key not in self.map:
            return -1
        node = self.map[key]
        self._remove(node)
        self._add_front(node)
        return node.val

    def put(self, key, value):
        if key in self.map:
            self._remove(self.map[key])
        node = DNode(key, value)
        self.map[key] = node
        self._add_front(node)
        if len(self.map) > self.capacity:
            lru = self.tail.prev
            self._remove(lru)
            del self.map[lru.key]

c = LRUCache(2)
c.put(1, 'a'); c.put(2, 'b')
print(c.get(1)) # 'a'
c.put(3, 'c')   # evicts key 2
print(c.get(2)) # -1
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: O(1) for get and put, guaranteed by direct pointer manipulation (no shifting like a list-based approach).
2. Edge case: capacity = 1 correctly evicts the sole existing entry whenever a new key is put.
3. Edge case: repeated put() calls on the same key update the value AND move it to the front without growing map size.
4. Edge case: get() on a missing key returns -1 without mutating the structure.
5. Pitfall: forgetting to update BOTH prev and next pointers on removal/insertion corrupts the linked list silently, causing subtle bugs that only appear after several operations.
6. Alternative: OrderedDict-based implementation (see Question 33) is simpler to write correctly but relies on the standard library's internal doubly-linked-list implementation rather than demonstrating it manually.

Q47. Word Ladder (Shortest Transformation, BFS)

HARD

Problem: Find the shortest transformation sequence length from a start word to an end word, changing one letter at a time, each intermediate word must be in a given word list.

```
from collections import deque

def word_ladder_length(begin, end, word_list):
    words = set(word_list)
    if end not in words:
        return 0
    queue = deque([(begin, 1)])
    visited = {begin}
    alphabet = "abcdefghijklmnopqrstuvwxyz"

    while queue:
        word, steps = queue.popleft()
        if word == end:
            return steps
        for i in range(len(word)):
            for ch in alphabet:
                candidate = word[:i] + ch + word[i+1:]
                if candidate in words and candidate not in visited:
                    visited.add(candidate)
                    queue.append((candidate, steps + 1))

    return 0

print(word_ladder_length("hit", "cog", ["hot","dot","dog","lot","log","cog"]))
# 5 ("hit" -> "hot" -> "dot" -> "dog" -> "cog")
print(word_ladder_length("hit", "xyz", ["hot","dot"]))
# 0 (end word not reachable / not in list)
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(N * L^2 * 26)$ where N = number of words, L = word length -- generating all one-letter variants per word.
2. Edge case: end word not present in word_list immediately returns 0 (no possible transformation).
3. Edge case: begin word equal to end word -- depending on requirements, this implementation would still need end in words to return anything other than 0; clarify if $begin == end$ should short-circuit to 1.
4. Edge case: no valid transformation path exists -- the BFS queue empties and the function returns 0.
5. Pitfall: not tracking 'visited' causes the BFS to revisit words repeatedly, potentially causing exponential blow-up or infinite loops in cyclic word graphs.
6. Alternative: bidirectional BFS (searching from both begin and end simultaneously) significantly reduces the search space for large dictionaries.

Q48. N-Queens Problem

HARD

Problem: Count the number of ways to place n queens on an $n \times n$ chessboard so that no two queens attack each other.

```
def solve_n_queens(n):
    solutions = 0
    cols, diag1, diag2 = set(), set(), set()

    def backtrack(row):
        nonlocal solutions
        if row == n:
            solutions += 1
            return
        for col in range(n):
            if col in cols or (row - col) in diag1 or (row + col) in diag2:
                continue
            cols.add(col); diag1.add(row - col); diag2.add(row + col)
            backtrack(row + 1)
            cols.remove(col); diag1.remove(row - col); diag2.remove(row + col)

    backtrack(0)
    return solutions

print(solve_n_queens(4)) # 2
print(solve_n_queens(1)) # 1
print(solve_n_queens(2)) # 0
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(n!)$ worst case due to backtracking search space, though pruning via sets makes it much faster in practice than brute force.
2. Edge case: $n = 1$ has exactly 1 trivial solution (a single queen on a 1×1 board).
3. Edge case: $n = 2$ and $n = 3$ have 0 solutions -- it's mathematically impossible to place non-attacking queens on boards that small (except $n=1$).
4. Edge case: $n = 0$ returns 1 by this implementation's logic ($row == n$ is immediately true, the 'empty placement' counts as one solution) -- decide if this convention matches requirements.
5. Pitfall: forgetting to remove elements from `cols/diag1/diag2` after backtracking (the 'undo' step) corrupts state for sibling branches, causing wildly incorrect counts.
6. Alternative: to also RECONSTRUCT the actual board layouts (not just count them), maintain a list of column placements per row and append a copy when $row == n$.

Q49. Coin Change (Minimum Coins)

HARD

Problem: Given coin denominations and a target amount, find the minimum number of coins needed to make that amount.

```
def coin_change(coins, amount):
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a and dp[a - c] + 1 < dp[a]:
                dp[a] = dp[a - c] + 1
    return dp[amount] if dp[amount] != float('inf') else -1

print(coin_change([1, 2, 5], 11)) # 3 (5+5+1)
print(coin_change([2], 3))       # -1 (impossible)
print(coin_change([1], 0))       # 0
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(\text{amount} * \text{len}(\text{coins}))$, space $O(\text{amount})$ for the DP array.
2. Edge case: amount = 0 requires 0 coins, returned immediately from $\text{dp}[0] = 0$.
3. Edge case: no combination of coins can make the amount (e.g. $\text{coins}=[2]$, $\text{amount}=3$) correctly returns -1.
4. Edge case: a coin denomination equal to the amount itself is handled naturally ($\text{dp}[\text{amount}]$ becomes 1).
5. Pitfall: using a greedy 'always pick the largest coin first' approach fails for non-canonical coin systems (e.g. $\text{coins}=[1,3,4]$, $\text{amount}=6$: greedy gives $4+1+1=3$ coins, but optimal is $3+3=2$ coins) -- DP is required for correctness.
6. Alternative: a BFS approach (treating amounts as graph nodes, coins as edges) also finds the minimum coin count and can be more intuitive to reason about.

Q50. TTL Memoization Decorator

HARD

Problem: Write a decorator that caches a function's results but expires each cached entry after a given number of seconds.

```
import time
import functools

def ttl_cache(ttl_seconds):
    def decorator(func):
        cache = {}
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            key = (args, tuple(sorted(kwargs.items())))
            now = time.monotonic()
            if key in cache:
                value, timestamp = cache[key]
                if now - timestamp < ttl_seconds:
                    return value
            result = func(*args, **kwargs)
            cache[key] = (result, now)
            return result
        wrapper.cache_clear = cache.clear
        return wrapper
    return decorator

@ttl_cache(ttl_seconds=2)
def slow_square(x):
    print(f"computing square of {x}")
    return x * x

print(slow_square(4)) # prints "computing..." then 16
print(slow_square(4)) # cached, no print, returns 16
time.sleep(2.1)
print(slow_square(4)) # expired, recomputes and prints again
```

6 Key Points (Edge Cases, Complexity & Notes)

1. Time complexity: $O(1)$ amortized per call for cache hits (dict lookup); space grows with the number of distinct argument combinations seen.
2. Edge case: calling with unhashable arguments (e.g. a list) raises a `TypeError` since dict keys must be hashable -- document this constraint clearly.
3. Edge case: kwargs order shouldn't matter for cache key equality -- handled by sorting `kwargs.items()` before building the key.
4. Edge case: an entry exactly at the ttl boundary (`now - timestamp == ttl_seconds`) is treated as expired since the check uses strict '`<`'.
5. Pitfall: using `time.time()` instead of `time.monotonic()` is vulnerable to system clock adjustments (e.g. NTP sync, daylight saving) causing incorrect expiry behavior.
6. Pitfall: this implementation is NOT thread-safe -- concurrent calls could race on cache reads/writes; a `threading.Lock` around cache access would be needed for multi-threaded use.